

Distribution Management System

Complete Technical & User Documentation

Version 1.0.0 | April 2026 | Confidential

FastAPI · React 18 · MySQL 8.4 · Docker · Tailscale

Table of Contents

- 1. [Overview](#)
 - 2. [System Architecture](#)
 - 3. [User Roles & Access Control](#)
 - 4. [Complete Workflows](#)
 - 5. [Features & Modules](#)
 - 6. [Database Design](#)
 - 7. [Security & Authentication](#)
 - 8. [API Reference](#)
 - 9. [UI/UX Structure](#)
 - 10. [Error Handling & Edge Cases](#)
 - 11. [Setup & Deployment](#)
 - 12. [Logging & Monitoring](#)
 - 13. [Future Improvements](#)
-

1. Overview

1.1 Introduction

The **Distribution Management System (DMS)** is a full-stack enterprise web application that manages the complete lifecycle of hardware device distribution across a multi-tier organizational hierarchy. The system enables PDIC (the central distribution body) to track every physical device — ONUs, ONTs, Routers, Modems, Set-top Boxes — from initial registration through every downstream hand-off to sub-distributors, cluster managers, and end-point operators. Every movement is confirmed, every defect is tracked, and every action is audited.

1.2 Objectives & Goals

Goal	Description
End-to-End Traceability	Every device is tracked from PDIC stock to the end operator and back
Structured Approvals	All device movements require confirmation; disputes are formally resolved
Defect Lifecycle Management	Formal defect → replacement → payment-due → closure workflow
Bulk Operations	Excel/CSV import for devices and distributions eliminates manual data entry
Hierarchical Access Control	Strict parent-child role scoping prevents unauthorized data access
Complete Audit Trail	Every API action is logged with actor identity, IP address, and timestamp

1.3 Problems It Solves

Problem	DMS Solution
No visibility of where devices are at any point	Full device tracking with serial number history timeline
Uncontrolled device hand-offs between staff	Structured distribution with mandatory receipt confirmation
No formal process for defective devices	Formal defect → replacement → confirmation workflow
Spreadsheet-based tracking prone to errors	Database-backed system with validation, bulk import, and real-time sync
No record of who did what or when	API activity log stored per action with role and IP
Users seeing data outside their scope	Role-based, hierarchy-scoped access enforced at route and service layer

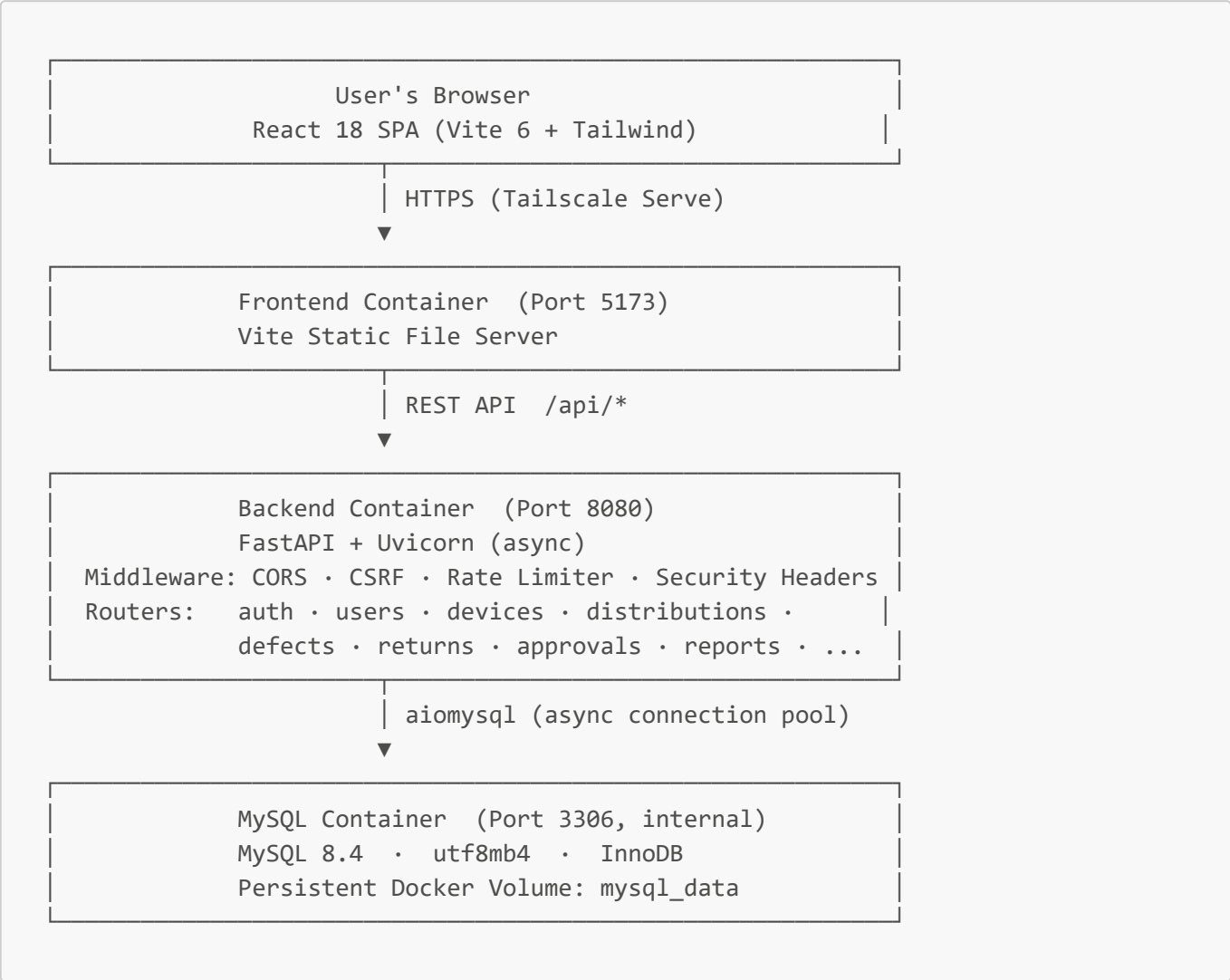
1.4 High-Level System Summary

Users access the system through a browser. After login, they receive a role-specific view of the system. Management staff at PDIC headquarters control the full device inventory, approve distributions, and manage defects. Sub-distributors, cluster managers, and operators exist in the field, receiving and redistributing devices within their hierarchy branch. The backend is a **FastAPI** application backed by **MySQL 8.4**. The frontend is a **React 18** SPA bundled with **Vite 6** and styled with **Tailwind CSS 3**. The full stack runs as Docker containers and is accessed privately via **Tailscale VPN**.

2. System Architecture

2.1 Overall Architecture

The system uses a **monolithic client-server architecture** with a clear separation between frontend and backend, both deployed via Docker Compose.



2.2 Component Breakdown

Frontend (frontend/)

File / Directory	Purpose
src/App.jsx	Root router, ProtectedRoute and PublicRoute guards
src/context/AuthContext.jsx	Global authentication state, token refresh loop
src/context/NotificationContext.jsx	Polling-based notification badge
src/pages/	37 page-level components
src/components/layout/	Sidebar, top navbar, responsive layout shell
src/services/api.js	Axios instance with base URL and interceptors

Backend (backend/app/)

Directory / File	Purpose
main.py	FastAPI app factory, middleware registration, router mounting
config.py	Pydantic settings loaded from .env — validated on startup
database.py	MySQL pool, table creation (19 tables), idempotent migrations
routes/	15 routers covering all API surface area
services/	Business logic — validation, state transitions, notifications
models/	Pydantic request/response schemas
middleware/	Auth middleware (get_current_user), error handler
core/	Rate limiter, audit logger, API activity logger

2.3 Request Data Flow

1. Browser → React component calls api.js (Axios)
2. Axios attaches HttpOnly cookie (JWT) automatically
3. Backend middleware validates JWT on every protected route
4. Route handler calls service layer
5. Service layer executes parameterized SQL via aiomysql pool
6. Response serialized as JSON → React state updated → UI re-renders
7. ApiActivityLoggingMiddleware logs actor + action to api_activity_logs

2.4 Third-Party Dependencies

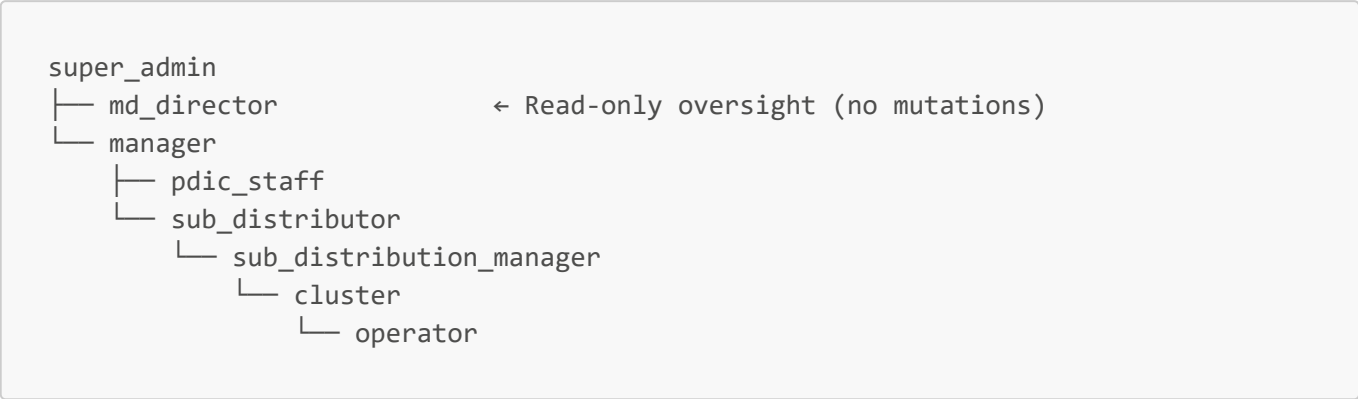
Dependency	Purpose
Tailscale	Private VPN tunnel + HTTPS termination for production
Docker / Docker Compose	Container orchestration
slowapi	Rate limiting on sensitive endpoints
starlette-csrf	CSRF token middleware
python-jose	JWT creation and validation (HS256)
passlib[bcrypt]	Password hashing — bcrypt with cost factor
aiomysql	Async MySQL driver
openpyxl / xlrd	Excel file parsing for bulk imports and manifest generation
lucide-react	Icon library
chart.js / react-chartjs-2	Dashboard charts and analytics
html5-qrcode	QR code scanner for device lookup

Dependency	Purpose
jspdf	PDF export of reports

3. User Roles & Access Control

The system defines **8 roles** in a strict hierarchy. Every user except `super_admin` has a `parent_id` pointing to their managing user within the tree.

3.1 Role Hierarchy



3.2 Role Definitions

`super_admin`

The highest privilege role with unrestricted access to the entire system.

Attribute	Detail
Create Users	Any role, including other super admins
Devices	Register, edit, delete, change status
Distributions	Create, approve, cancel, confirm returns
Defects	Full management — resolve, replace, confirm payment
Reports / Logs	All reports and the full API activity log
Restrictions	Cannot delete their own account
Typical Use	System setup, onboarding managers, emergency overrides

`md_director`

Managing Director / Director level. A **read-only observer** across the entire platform.

Attribute	Detail
Read Access	Users (except other super admins), devices, distributions, defects, reports
Write Access	None — all mutation routes explicitly block this role
Accessible Pages	Dashboard, Devices, Distributions, Defects, Reports, Activities, Backup
Typical Use	Executive reviewing distribution statistics and defect trends

 manager

Operational manager at PDIC level. Second highest effective privilege.

Attribute	Detail
Create Users	pdic_staff, sub_distribution_manager, sub_distributor, cluster, operator
Distributions	Create, approve/reject, confirm disputed returns
Defects	Acknowledge, update status, assign replacements, confirm payment
Returns	Approve and mark received
Change Requests	Review and approve credential/status change requests
Restrictions	Cannot create super_admin or md_director
Typical Use	Approving pending distributions, handling defect escalations

 pdic_staff

PDIC operations staff responsible for day-to-day device management.

Attribute	Detail
Devices	Register (single and bulk), view all PDIC stock
Distributions	Create distributions from PDIC to sub-level users
Users	View only (limited to own profile for non-role-specific queries)
Restrictions	Cannot approve distributions or manage users
Typical Use	Receiving a shipment of ONUs and bulk-importing them

 sub_distributor

External distributor who receives devices from PDIC and redistributes within their branch.

Attribute	Detail
Devices	View own held devices
Distributions	Create to sub_distribution_managers, clusters, or operators within their branch
Defects	View defects from their operators; forward to management
Delivery Confirmations	Confirm/dispute incoming distributions
Restrictions	Cannot access admin pages, approve, or register devices
Typical Use	Distributing a batch of routers to cluster managers

● sub_distribution_manager

Internal manager within a sub-distributor's branch overseeing clusters and operators.

Attribute	Detail
Users	Create and manage clusters and operators within their branch
Distributions	Can distribute held devices to clusters and operators under them
Restrictions	Cannot create distributions directly via bulk upload
Typical Use	Managing a group of cluster managers for a specific geographic area

○ cluster

Mid-level field user who holds devices and distributes to operators directly below.

Attribute	Detail
Distributions	Create to operators directly under their cluster
Defects	Report defects on their held devices
Delivery Confirmations	Confirm/dispute incoming distributions
Replacement Confirmation	Confirm receipt of a replacement device
Restrictions	Cannot access user management, reports, or approvals pages
Typical Use	A cluster manager distributing modems to 5 field operators in their area

● operator

End-level field user. The final recipient of devices in the chain.

Attribute	Detail
Devices	View own held devices
Defects	Create defect reports on their devices; send replacement enquiries
Delivery Confirmations	Confirm/dispute incoming distributions
Replacement Confirmation	Confirm receipt of replacement device
Pending Dues	View and track their outstanding payment obligations
Restrictions	Cannot access management pages, reports, or approvals
Typical Use	Field operator receiving a modem and later reporting it as defective

4. Complete Workflows

4.1 User Journey — From Login to Full Usage

Step 1: Navigate to /login

↓

Step 2: Enter email + password

→ Backend: normalize email, verify bcrypt hash, check account status

→ Rate limited: 5 attempts/minute per IP

→ Failed attempts tracked; account locks after threshold

↓

Step 3: Token issuance

→ access_token (15 min, HttpOnly cookie)

→ refresh_token (7 days, HttpOnly cookie)

→ CSRF token set

↓

Step 4: Check forced credential update flags

→ force_email_change = 1 → redirect to /force-update-credentials

→ force_password_change = 1 → redirect to /force-update-credentials

→ Both flags = 0 → proceed normally

↓

Step 5: Force-Update Screen (first login only)

→ User sets new email + new password

→ Current password verified before any change

→ New tokens issued, flags cleared

↓

Step 6: Dashboard loads

→ Role-specific KPI cards and charts

↓

Step 7: Core operations (role-dependent)

→ Admin/Manager: Approve items, manage users, register devices

→ Staff: Register devices, create distributions

→ Sub-distributor/Cluster/Operator: Confirm deliveries, report defects

↓

Step 8: Logout

→ POST /api/auth/logout

→ Token blacklisted in DB

→ Cookies cleared

4.2 Distribution Flow — Full Lifecycle

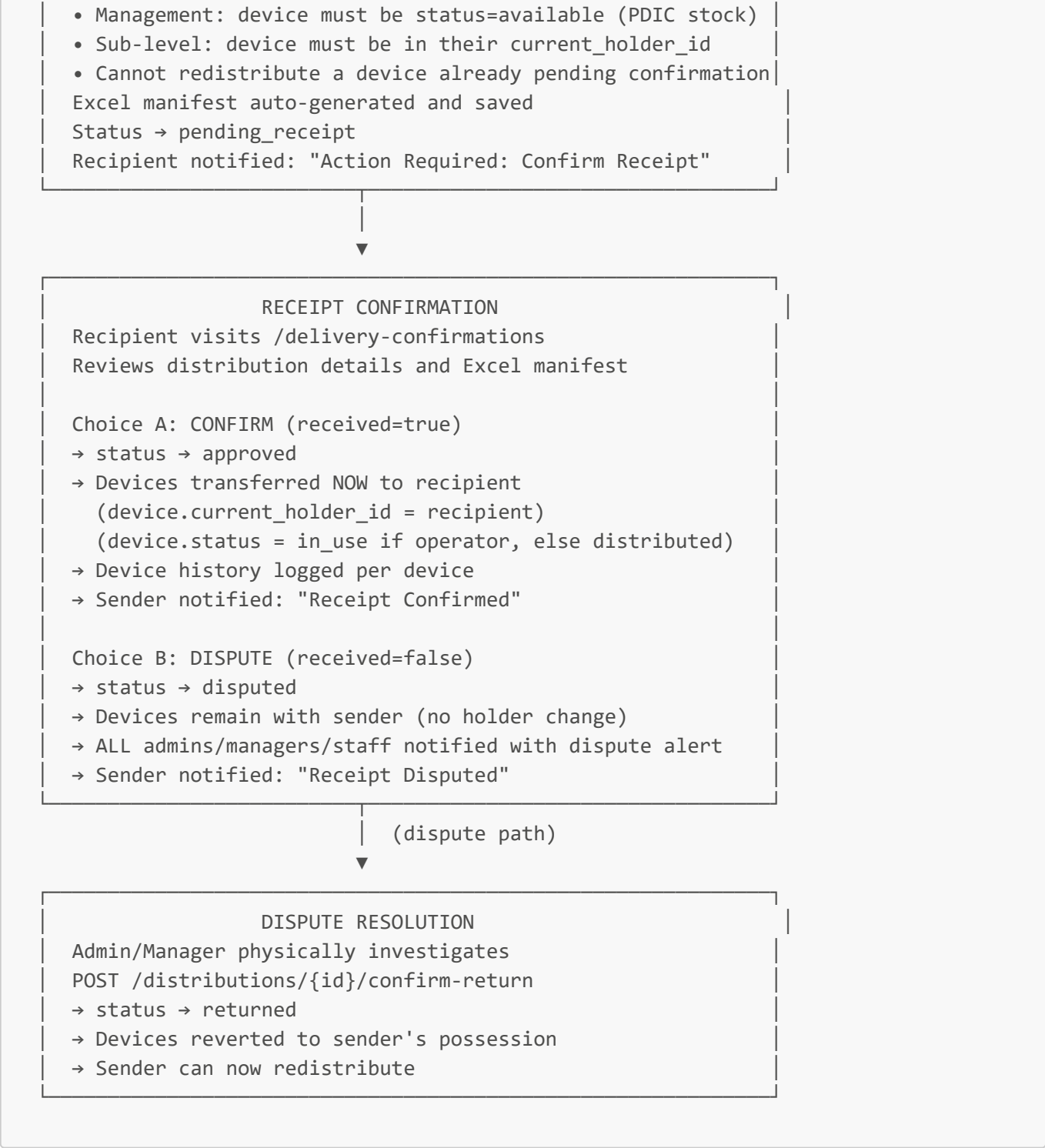
The distribution flow is the core operational workflow. Devices move from sender to recipient only **after the recipient explicitly confirms receipt**.

CREATION

Sender selects recipient + devices → POST /distributions

• Hierarchy validated (role-based cross-check)

• Each device checked: not defective, not locked



Hierarchy Validation Rules (Enforced at Service Layer)

Sender Role	Allowed Recipients
super_admin / manager / pdic_staff	sub_distributor, cluster, operator
sub_distributor	sub_distribution_manager or clusters/operators directly in their branch
sub_distribution_manager	clusters and operators directly under them
cluster	operators directly under their cluster

Sender Role	Allowed Recipients
operator	other operators in the same cluster only

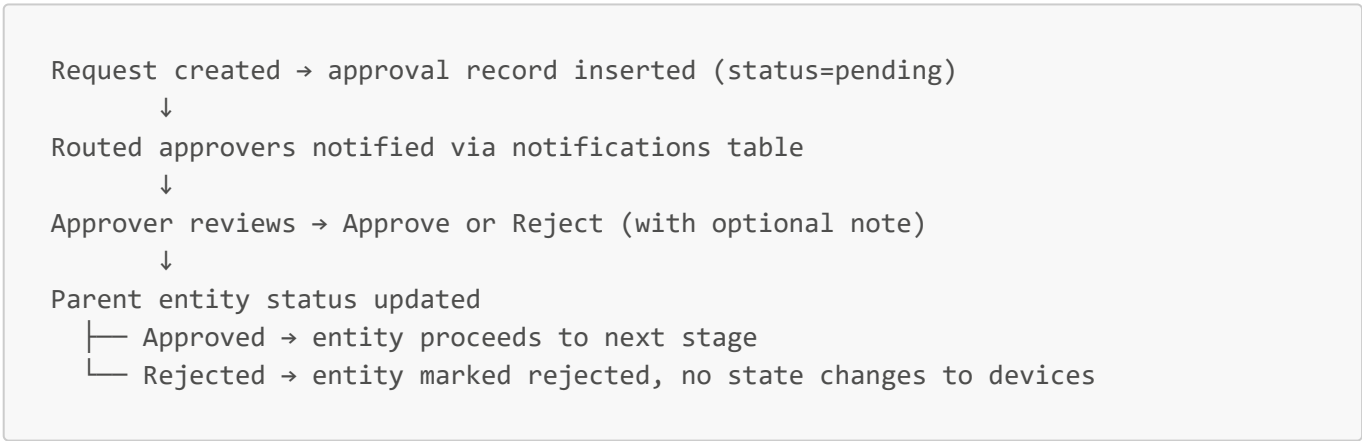
Bulk Distribution via File Upload

1. Upload CSV/XLSX with columns `mac_address` and/or `nuid`
2. System resolves each row to a device record via MAC or NUID lookup (case-insensitive)
3. If MAC and NUID both present and resolve to **different** devices → row error
4. If **any** row has an error → **entire upload is rejected** with per-row error list
5. If all rows valid → distribution created as above

4.3 Approval Flow

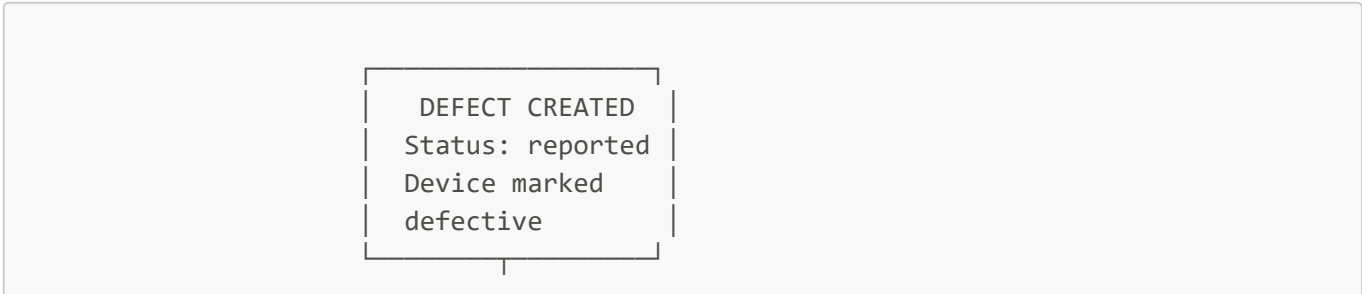
The `approval_role_routing` table governs which roles are authorized to process each approval type. This is configurable by super admins.

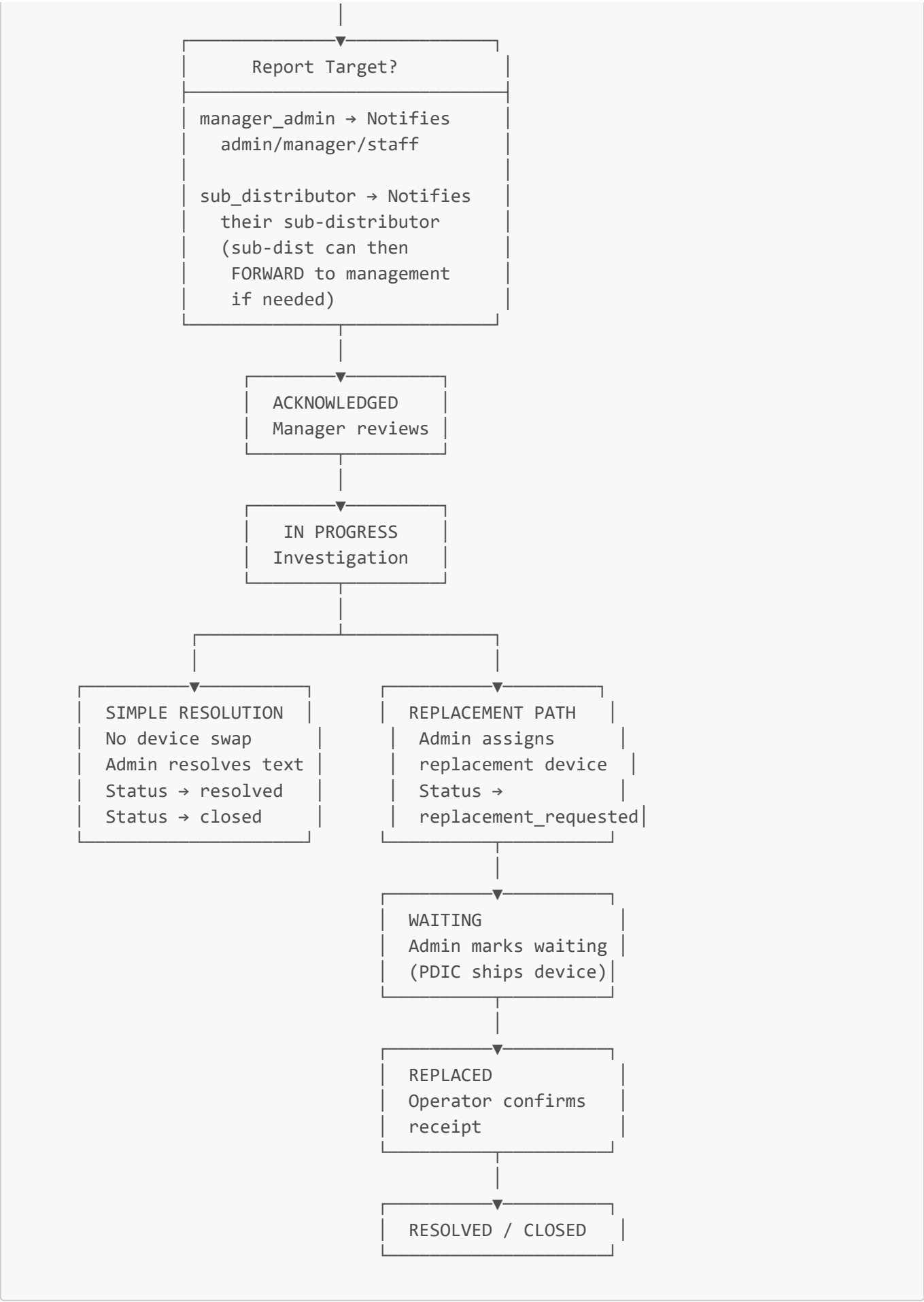
Approval Type	Default Approvers
<code>distribution</code>	super_admin, manager, pdic_staff
<code>return</code>	super_admin, manager, pdic_staff
<code>defect</code>	super_admin, manager, pdic_staff



Note: If the `staff_enabled` flag for an approval type is turned off in `approval_role_routing`, pdic_staff will be blocked from processing that type of approval even if they attempt it. The system raises a `PermissionError` checked at service level.

4.4 Defect Reporting Flow — Complete Lifecycle





Defect Status Values

Status	Description
reported	Initial state when defect is created
acknowledged	Management has acknowledged the report
in_progress	Under active investigation
replacement_requested	Admin has assigned a replacement device
waiting_for_replacement	Replacement being shipped from PDIC
replaced	Operator confirmed receipt of replacement
resolved	Defect fully resolved (with or without replacement)
closed	Final terminal state

Defect Routing: sub_distributor target

Only operators can route to `sub_distributor`. The system resolves the operator's sub-distributor by walking their parent chain:

- Operator → parent Cluster → parent Sub-distributor
- Operator → parent Sub-distributor (if directly under one)

Sub-distributor can then call `POST /defects/{id}/forward-to-management` to escalate to the manager/admin queue.

4.5 Replacement & Resolution Flow — Detailed Steps

Step 1: Admin calls `POST /defects/{id}/replace`
Provide ONE of:

- a) `replacement_device_id` (existing device by DB id)
- b) `mac_address` (system looks up device)
- c) `serial_number` (system looks up device)
- d) `register_device {}` (creates new device on the fly)

Optional: `return_amount` (financial charge), `notes`
↓

Step 2: System actions on replacement

- Defective device: `status` → defective (already set)
- `replacement_device_id` stored on defect record
- Replacement device: `current_holder_id` → operator
- Device history logged for replacement device
- Auto-return record created for defective device tracking
- `Status` → `replacement_requested`
- Operator notified: "Replacement Device Assigned"

↓

Step 3 (optional): Admin marks waiting
`POST /defects/{id}/mark-waiting`
`Status` → `waiting_for_replacement`
"PDIC is processing shipment of replacement"

```

↓
Step 4: Operator confirms receipt
POST /defects/{id}/replacement/confirm
Status → replaced
replacement_confirmed_at and replacement_confirmed_by set
↓
Step 5 (if return_amount > 0): Payment flow
Admin uploads bill: POST /defects/{id}/payment-bill
(JPG / PNG / WEBP / PDF ≤ 8MB)
bill_url stored; operator notified of pending payment
↓
Admin confirms payment: POST /defects/{id}/confirm-payment
All conditions must pass:
  • return_amount > 0
  • payment not already confirmed
  • auto-return status must be "received" first
payment_confirmed = 1; operator notified: "Payment Confirmed"
↓
Step 6: Admin resolves
PATCH /defects/{id}/resolve
Status → resolved (then closed)

```

If Operator Doesn't Receive Replacement

- Operator sends **enquiry**: `POST /defects/{id}/enquire` with message
- Management notified with operator's message
- Admin can **resend confirmation notification**: `POST /defects/{id}/resend-confirmation`

4.6 Return Flow

```

User requests return:
POST /api/returns
Reasons: defective | excess_stock | wrong_device | end_of_contract | other
Status → pending
↓
Admin/Manager approves:
PATCH /api/returns/{id}/approve
Status → approved
↓
Physical device received at PDIC:
PATCH /api/returns/{id}/receive
Status → received
Device status → returned / available

```

When a defect is **approved** (`update_defect_status` with `approved`), an auto-return record is created automatically linking to the defect report via `auto_return_id`. This ensures the return is tracked without requiring a separate manual request.

5. Features & Modules

5.1 Authentication Module

Routes: `POST /api/auth/login`, `/logout`, `/refresh`, `/me`, `/password`, `/complete-forced-update`

Field	Detail
Method	JWT (HS256) stored in HttpOnly cookies
Tokens	access_token (15 min), refresh_token (7 days)
Rate Limit	5 login attempts/min per IP
CSRF	starlette-csrf middleware; login endpoint exempt
Brute Force	Failed attempts tracked; <code>locked_until</code> set on threshold
Forced Update	First-login flags cleared after user sets new credentials

Login Error States:

- Wrong credentials → `401 Invalid email or password`
- Inactive account → `403 Account is not active`
- Locked account → `401` (same message, timing-safe)

5.2 Device Registration

Routes: `POST /api/devices`, `POST /api/devices/bulk-upload`

Field	Detail
Access	super_admin, manager, pdic_staff
Single Registration	device_type, model, serial_number, mac_address, manufacturer, band_type, nuid
Bulk Upload	Excel (.xlsx/.xls) or CSV — two supported schemas
File Validation	Magic byte check (PK for XLSX, D0CF for XLS, no null bytes for CSV)
Size Limit	10 MB maximum
On Create	device_id auto-generated (<code>DEV-{uuid}</code>), status=available, history logged

Bulk Upload Schemas:

Schema	Required Columns
Regular (ONU/Router/Modem)	<code>vendor</code> , <code>device_type</code> , <code>model</code> , <code>mac_address</code> , <code>serial_number</code>
Set-top Box (SB)	<code>vendor</code> , <code>device_type</code> , <code>model</code> , <code>nuid</code> , <code>box_type</code> (<code>box_type</code> must be HD or OTT)

Partial failures are reported per row without aborting the entire upload.

5.3 Device Tracking

Route: GET /api/devices/track/{serial_number}

Provides full lifecycle visibility of any device. The frontend at /devices/track includes a **QR code scanner** (html5-qrcode) allowing field staff to scan a device label and instantly retrieve its history.

History Entry Actions: registered, distributed, returned, defective, replaced, status_change, holder_updated

5.4 Distribution Management

Routes: GET/POST /api/distributions, GET /api/distributions/{id}, receipt, status, cancel, manifest, bulk-upload

Feature	Detail
Manifest	Auto-generated Excel file per distribution with all device details
MAC/NUID Export	Download device list as CSV or XLSX for reference
Bulk Upload	Create distribution from CSV/XLSX using mac_address/nuid lookup
Cancellation	Only the creator can cancel a pending distribution
Scoping	Sub-level users see only distributions they sent or received

5.5 Defect Reports

Routes: GET/POST /api/defects, full lifecycle endpoints

Feature	Detail
Defect Types	hardware, software, connectivity, physical_damage, other
Severity Levels	critical, high, medium, low
Images	Multiple JPG/PNG/WEBP files, stored in /uploads/
Payment Bill	JPG/PNG/WEBP/PDF ≤ 8MB uploaded as proof of payment
Visibility Scoping	Operators see own defects; clusters see branch; management sees all
Duplicate Prevention	Cannot create a new defect for a device that has an active open defect

5.6 Returns Management

Routes: GET/POST /api/returns, approve, receive

Feature	Detail
Return Reasons	defective, excess_stock, wrong_device, end_of_contract, other

Feature	Detail
Statuses	pending → approved → received
Auto-Return	Created automatically when a defect is approved
MAC Tracking	<code>mac_address</code> field stored for identification without device DB lookup

5.7 User Management

Routes: `GET/POST/PUT/DELETE /api/users`, status, credentials, role filter

Feature	Detail
Hierarchy Enforcement	Parent-child assignment validated at creation and update
Force Flags	<code>force_email_change</code> , <code>force_password_change</code> cleared after first login
Status	active / inactive / suspended
Credential Reset	Super admin can reset any user email/password via admin endpoint
Branch Traversal	<code>_branch_contains_user()</code> recursively validates scoped access
Hierarchy View	<code>/users/hierarchy</code> renders tree visualization

Creation Permissions:

Actor	Can Create
super_admin	All roles including super_admin
manager	pdic_staff, sub_distribution_manager, sub_distributor, cluster, operator
sub_distribution_manager	cluster, operator

5.8 Notifications

Routes: `GET /api/notifications`, mark read, mark all read

In-app notification system with per-user scoping. Notifications are written to the `notifications` table by service-layer events (not by a separate daemon).

Trigger	Recipient
Distribution created	Recipient user
Receipt confirmed	Sender
Receipt disputed	All admins/managers/staff + Sender
Defect created	Admin/manager/staff (or sub-distributor if targeted)
Defect approved	Reporter, management staff for return approval

Trigger	Recipient
Replacement assigned	Operator
Payment confirmed	User with due amount

UI: Bell icon with unread count badge, dropdown showing latest 5, full list at </notifications>.

5.9 Change Requests

Routes: [GET/POST /api/change-requests](#), approve, reject

Users can request credential or device status changes. Admins review and approve or reject.

Request Type	Submitter	Reviewer
Email change	Any user	super_admin, manager
Password change	Any user	super_admin, manager
Device status change	pdic_staff	super_admin, manager

Security note: Passwords in pending change requests are stored **hashed** immediately (bcrypt). A boot-time migration re-hashes any legacy plaintext values found in the database.

5.10 External Inventory

Routes: [GET/POST /api/external-inventory](#) (items, purchase orders, receipts, movements)

A supplementary inventory system for non-system items (spare parts, accessories, consumables).

Entity	Purpose
external_inventory_items	Physical stock records with SKU, price, quantity
inventory_purchase_orders	POs raised to suppliers
inventory_po_lines	Line items per PO
inventory_receipts	Goods received against a PO
inventory_stock_movements	Every stock in/out event

5.11 Reports & Analytics

Routes: [GET /api/reports](#)

Report Type	Filters Available
Distribution Report	Date range, status, user
Device Inventory Report	Device type, status, holder

Report Type	Filters Available
Defect Report	Severity, type, status, date range

Reports can be exported as **PDF** (via [jsPDF](#)) or printed directly from the browser.
Access: super_admin, md_director, manager, pdic_staff

5.12 Activity Log (Audit Trail)

Route: [GET /api/activities](#) (via [api_activity_logs](#) table)

Every significant API request is logged automatically by [ApiActivityLoggingMiddleware](#) on every response.

Captured Field	Example
actor_id	"42"
actor_name	"John Manager"
actor_role	"manager"
method	"POST"
path	"/api/distributions"
status_code	201
ip_address	"100.64.1.5"
description	"Distribution created"
created_at	"2026-04-07T15:00:00"

Excluded from logging: OPTIONS preflight, [/health](#) check, insignificant GETs.
Access: super_admin, md_director only.

5.13 Backup

Route: [GET /api/reports/backup](#) (download), scheduled background task

Feature	Detail
Manual Backup	Admin/Manager/MD can trigger and download backup
Scheduled Backup	Monthly background scheduler (backup_scheduler_loop) runs on startup
Storage	backend/monthly_backups/ directory (mounted Docker volume)
Retention	Files persist in Docker volume across restarts

5.14 Delivery Confirmations

Route: [GET /delivery-confirmations](#) (frontend page calling distributions API)

Dedicated page for receiving users to confirm or dispute incoming distributions. Shows only distributions where `to_user_id = current_user.id` and `status = pending_receipt`.

Actions available:

- **Confirm Receipt** → triggers device transfer to recipient's account
 - **Dispute Receipt** → triggers admin/manager notification and dispute workflow
-

5.15 Pending Dues

Routes: `GET /api/defects/pending-dues/me` (field users), `GET /api/defects/pending-dues/users` (management)

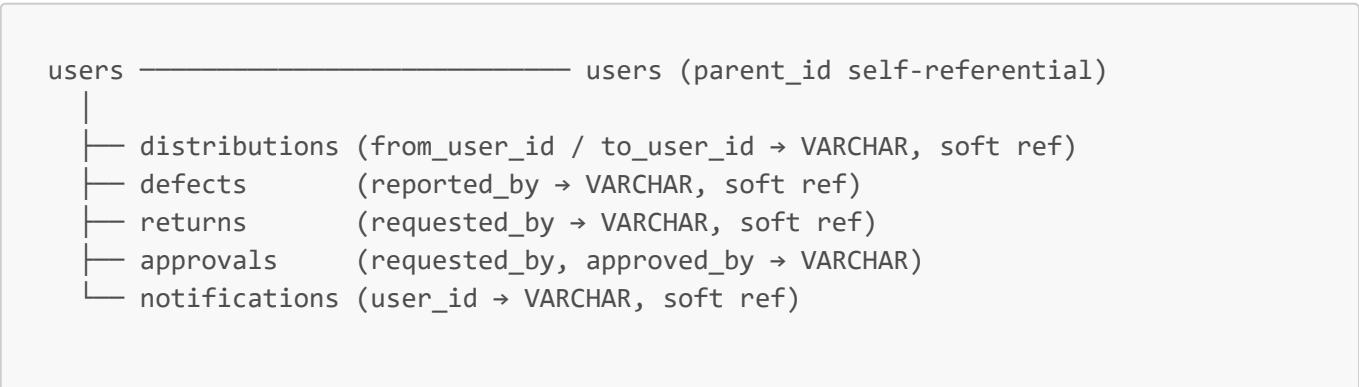
Tracks unresolved financial obligations when a defective device return carries a `return_amount`. A due is cleared when `payment_confirmed = 1`.

6. Database Design

6.1 Tables Overview

Table	Rows / Nature	Purpose
users	Core entity	All system users with role, hierarchy, auth state
devices	Core entity	Hardware devices with full lifecycle status
device_history	Append-only	Immutable audit log of every device state change
distributions	Transaction	Distribution requests and their complete lifecycle
defects	Transaction	Defect reports with replacement and payment lifecycle
returns	Transaction	Return requests for defective / excess devices
approvals	Junction	Generic approval records for all approvable entities
operators	Reference	External operator contact directory
notifications	Inbox	Per-user notification messages
change_requests	Request	User-submitted credential or device status change requests
external_inventory_items	Inventory	Non-system stock items
inventory_purchase_orders	Inventory	POs for external inventory procurement
inventory_po_lines	Inventory	Line items on each PO
inventory_receipts	Inventory	Goods receipts against POs
inventory_receipt_lines	Inventory	Line items on each receipt
inventory_stock_movements	Inventory	Every stock movement event
api_activity_logs	Audit	Full API call audit trail
approval_role_routing	Config	Role → approval type permission map
token_blacklist	Security	Revoked JWT tokens

6.2 Key Relationships



```

devices ----- device_history (device_id → VARCHAR)
|
└─ defects (device_id → VARCHAR, soft ref)
    |
    └─ returns (defect_id → VARCHAR via auto_return_id)

```

All foreign-key-like references are stored as **VARCHAR** rather than SQL FK constraints. This is intentional to allow soft-delete and role-type tracking without cascading complexity. Referential integrity is enforced at the **service layer**.

6.3 Critical Table Schema Details

users

id	INT AUTO_INCREMENT PRIMARY KEY	
email	VARCHAR(255) UNIQUE NOT NULL	
name	VARCHAR(255) NOT NULL	
password_hash	TEXT NOT NULL	-- bcrypt hash, never plaintext
role	VARCHAR(64) NOT NULL	-- one of 8 role values
parent_id	INT NULL	-- points to managing user
force_email_change	TINYINT(1) DEFAULT 0	-- forces update on next login
force_password_change	TINYINT(1) DEFAULT 0	-- forces update on next login
failed_login_attempts	INT DEFAULT 0	-- incremented on bad password
locked_until	VARCHAR(64)	-- ISO timestamp of lock expiry
status	VARCHAR(32)	-- active / inactive / suspended
permissions	LONGTEXT	-- JSON blob, custom overrides
theme	VARCHAR(32)	-- light / dark / system
compact_mode	TINYINT(1) DEFAULT 0	

devices

id	INT AUTO_INCREMENT PRIMARY KEY	
device_id	VARCHAR(128) UNIQUE	-- DEV-{uuid}
serial_number	VARCHAR(255) UNIQUE	
mac_address	VARCHAR(255) UNIQUE	
status	VARCHAR(64)	-- available / distributed / in_use -- defective / returned / replaced
current_holder_id	VARCHAR(64)	-- user id holding the device
current_holder_name	VARCHAR(255)	
current_holder_type	VARCHAR(64)	-- noc / sub_distributor / cluster /
operator		
band_type	VARCHAR(64)	-- single_band / dual_band
nuid	VARCHAR(255)	-- Set-top box unique ID
metadata	LONGTEXT	-- JSON (e.g., {"box_type": "HD"})

distributions

distribution_id	VARCHAR(128) UNIQUE	-- DIST-{uuid}
device_ids	LONGTEXT	-- JSON array of device IDs
device_count	INT	
from_user_id	VARCHAR(64)	
to_user_id	VARCHAR(64)	
status	VARCHAR(64)	-- pending_receipt / approved /
disputed		-- returned / cancelled / rejected
manifest_file	VARCHAR(255)	-- filename of Excel manifest
approval_date	VARCHAR(64)	-- set when recipient confirms

defects

report_id	VARCHAR(128) UNIQUE	-- DEF-{uuid}
device_id	VARCHAR(64)	
defect_type	VARCHAR(64)	-- hardware / software / connectivity
		-- physical_damage / other
severity	VARCHAR(64)	-- critical / high / medium / low
report_target	VARCHAR(64)	-- manager_admin / sub_distributor
forwarded_to_management	TINYINT(1)	-- 1 if sub-dist forwarded to mgmt
status	VARCHAR(64)	-- reported → acknowledged → ...
replacement_device_id	VARCHAR(64)	-- ID of assigned replacement
auto_return_id	VARCHAR(64)	-- linked return_id for device return
return_amount	DOUBLE DEFAULT 0	-- financial charge for damage
payment_bill_url	VARCHAR(255)	-- path to uploaded bill
payment_confirmed	TINYINT(1) DEFAULT 0	
operator_id	VARCHAR(64)	-- resolved during report creation
sub_distributor_id	VARCHAR(64)	-- resolved during report creation

7. Security & Authentication

7.1 Authentication Model

Component	Implementation
Algorithm	HS256 JWT via <code>python-jose</code>
Storage	HttpOnly cookies — never localStorage or sessionStorage
Access Token	15-minute expiry, validated on every protected route
Refresh Token	7-day expiry, used to silently re-issue access tokens
Revocation	Blacklist token hash in <code>token_blacklist</code> table on logout
Cookie flags	<code>HttpOnly=true</code> , <code>SameSite=Strict</code> , <code>Secure=true</code> in production

Token Refresh Cycle (Frontend):

```
Request → 401 Unauthorized
  → AuthContext intercepts
  → POST /api/auth/refresh (using refresh cookie)
  → New access_token set in cookie
  → Original request retried automatically
  → If refresh also fails → redirect to /login
```

7.2 Authorization Layers

Authorization is enforced at **two levels** for defense in depth:

Layer 1 — Route Level:

- `Depends(get_current_user)` — validates JWT and loads user object
- `Depends(require_admin_or_manager)` — blocks non-management
- `Depends(require_management)` — allows admin, manager, staff
- `Depends(require_any_role)` — any authenticated user
- `_ensure_not_md_director()` — blocks MD/Director from mutations

Layer 2 — Service Level:

- Hierarchy traversal (`_branch_contains_user()`) for write access
- Scope filtering (only show data within user's hierarchy branch)
- Business rule validation (device ownership, distribution constraints, etc.)

7.3 Security Controls Summary

Control	Implementation
Rate Limiting	slowapi: 5/min login · 10/min token refresh · 30/min logout

Control	Implementation
CSRF Protection	starlette-csrf middleware on all state-changing requests
Security Headers	X-Content-Type-Options, X-Frame-Options: DENY, X-XSS-Protection, Referrer-Policy, Permissions-Policy, Content-Security-Policy
HTTPS Enforcement	<code>ENFORCE_HTTPS=true</code> → 307 redirect; HSTS header added in production
Password Hashing	bcrypt via passlib — never stored plaintext
SQL Injection	Parameterized queries throughout (<code>?</code> → <code>%s</code> translated for MySQL)
File Upload	Magic byte validation, extension allowlist, size caps (10MB / 8MB)
Path Traversal	Uploads served via route that resolves and checks path within root
Sensitive Data	<code>password_hash</code> stripped from all API responses
Account Lockout	<code>failed_login_attempts</code> tracked; <code>locked_until</code> set on threshold
Audit Logging	All API actions logged with actor id, role, IP, status code
Docs Hidden	Swagger UI disabled when <code>DEBUG=false</code> (production)

7.4 SECRET_KEY Validation

At application startup, `Settings` validates the secret key:

- Must be $\geq$ **32 characters**
- Must **not** contain the string `dms` (prevents weak defaults)
- If invalid → **application refuses to start**

```
# Generate a secure key:
python -c "import secrets; print(secrets.token_urlsafe(64))"
```

8. API Reference

8.1 Base URL & Standard Format

Base URL: `http(s)://<host>:8080/api`

Success Response:

```
{
  "success": true,
  "message": "Human-readable description",
  "data": { ... },
  "pagination": {
    "page": 1, "page_size": 20,
    "total": 150, "total_pages": 8,
    "has_next": true, "has_prev": false
  }
}
```

Error Response:

```
{
  "detail": "Descriptive error message (never exposes stack traces)"
}
```

8.2 HTTP Status Codes

Code	Meaning
200	Success
201	Resource created
400	Bad request / validation failure
401	Not authenticated
403	Authenticated but lacks permission
404	Resource not found
413	File too large
422	Request body schema validation failure
429	Rate limit exceeded
500	Internal server error (sanitized message only)

8.3 Authentication Endpoints

Method	Path	Description	Auth
POST	/api/auth/login	Login with email + password	Public
POST	/api/auth/logout	Logout + blacklist token	Required
POST	/api/auth/refresh	Issue new access token from refresh cookie	Cookie
GET	/api/auth/me	Get current user's profile	Required
PUT	/api/auth/password	Change own password	Required
POST	/api/auth/complete-forced-update	First-login credential rotation	Required

Sample Login Request:

```
POST /api/auth/login
Content-Type: application/json

{ "email": "manager1@dms.com", "password": "Manager@123" }
```

Sample Login Response:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "access_token": "<jwt>",
    "refresh_token": "<jwt>",
    "token_type": "bearer",
    "user": { "id": "5", "name": "Manager One", "role": "manager" }
  }
}
```

8.4 Device Endpoints

Method	Path	Description	Min Role
GET	/api/devices	List devices (scoped by role)	Any
POST	/api/devices	Register single device	staff
GET	/api/devices/{id}	Get device by ID	Any
PUT	/api/devices/{id}	Update device	manager
DELETE	/api/devices/{id}	Delete device	manager

Method	Path	Description	Min Role
PATCH	/api/devices/{id}/status	Update status	Any
GET	/api/devices/{id}/history	Full device history	Any
GET	/api/devices/track/{serial}	Track by serial number	Any
GET	/api/devices/available	Devices available to distribute	Any
GET	/api/devices/for-replacement	Replacement-eligible pool	staff
GET	/api/devices/my-overview	Dashboard device stats	Any
POST	/api/devices/bulk-upload	Bulk register from file	staff
POST	/api/devices/{id}/request-edit	Staff submits edit request	staff
POST	/api/devices/{id}/repair-holder	Admin repairs holder from history	manager

8.5 Distribution Endpoints

Method	Path	Description	Min Role
GET	/api/distributions	List distributions	Any
POST	/api/distributions	Create distribution	operator+
GET	/api/distributions/{id}	Get distribution	Any
PATCH	/api/distributions/{id}/status	Update status	Any
DELETE	/api/distributions/{id}	Cancel	Creator only
POST	/api/distributions/{id}/receipt	Confirm or dispute receipt	Recipient only
POST	/api/distributions/{id}/confirm-return	Confirm disputed return	manager
GET	/api/distributions/{id}/manifest	Download Excel manifest	Any
GET	/api/distributions/{id}/export-mac-nuid	Export MAC/NUID CSV or XLSX	Any
POST	/api/distributions/bulk-upload	Create distribution from file	operator+
GET	/api/distributions/pending	Pending distributions	management
POST	/api/distributions/sync-devices	Admin device sync	manager

Sample Create Distribution:

```
POST /api/distributions
Content-Type: application/json

{
```

```
"device_ids": ["101", "102", "103"],
"to_user_id": "42",
"notes": "Q1 2026 modem deployment"
}
```

Sample Response:

```
{
  "success": true,
  "message": "Distribution created successfully",
  "data": {
    "distribution_id": "DIST-abc123",
    "status": "pending_receipt",
    "device_count": 3,
    "from_user_name": "PDIC Staff",
    "to_user_name": "Operator A",
    "request_date": "2026-04-07T15:00:00",
    "manifest_file": "DIST-abc123-devices.xlsx"
  }
}
```

8.6 Defect Endpoints

Method	Path	Description	Min Role
GET	/api/defects	List defects (scoped)	Any
POST	/api/defects	Create defect report	Any
GET	/api/defects/{id}	Get defect by ID	Any
PUT	/api/defects/{id}	Update defect	manager
DELETE	/api/defects/{id}	Delete defect	manager
PATCH	/api/defects/{id}/status	Update status	management
PATCH	/api/defects/{id}/resolve	Resolve defect	manager
POST	/api/defects/{id}/replace	Assign replacement device	management
POST	/api/defects/{id}/replacement/confirm	Operator confirms replacement	Any
POST	/api/defects/{id}/enquire	Send replacement enquiry	operator/cluster/sub-dist
POST	/api/defects/{id}/resend-confirmation	Resend confirmation to operator	management

Method	Path	Description	Min Role
POST	/api/defects/{id}/mark-waiting	Mark as waiting for shipment	management
POST	/api/defects/{id}/forward-to-management	Forward routed defect upward	sub_distributor
POST	/api/defects/{id}/payment-bill	Upload payment bill file	manager
POST	/api/defects/{id}/confirm-payment	Confirm payment received	manager
GET	/api/defects/replacements	All active replacement mappings	Any
GET	/api/defects/replacements/pending	Defects awaiting replacement	Any
GET	/api/defects/pending-dues/users	Users with pending dues	management
GET	/api/defects/pending-dues/me	My pending dues	operator/cluster/sub-dist

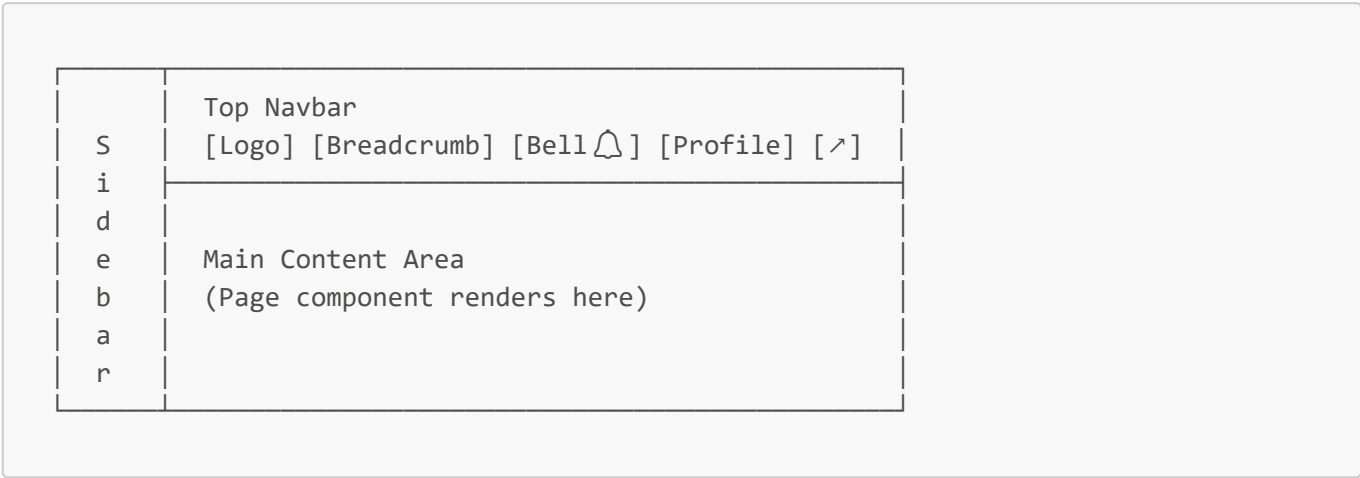
8.7 Other Endpoint Groups

Prefix	Key Operations
/api/users	CRUD users, update status, reset credentials, get by role
/api/returns	Create, approve, mark received
/api/approvals	List, approve, reject, configure routing
/api/operators	CRUD external operators
/api/notifications	List, mark read, mark all read
/api/reports	Generate and export reports
/api/dashboard	Role-specific KPI stats
/api/change-requests	CRUD, approve, reject
/api/external-inventory	Items, POs, receipts, stock movements
/health	Health check (no auth)

9. UI/UX Structure

9.1 Layout Shell

The layout is composed of three zones:



- **Sidebar** — Role-gated navigation links, collapsible on mobile
- **Navbar** — User name, role badge, notification bell with unread count, logout
- **Theme** — Light / Dark / System — stored per user in DB, applied via CSS class

9.2 All Application Pages

Route	Page	Key UI Elements
/login	Login	Email/password form, error states
/force-update-credentials	Force Update	Mandatory new email + password form
/	Dashboard	Role-specific KPI cards, distribution/defect charts
/devices	Devices	Filterable table, status badges, device detail modal
/devices/register	Register Device	Form with device type selector, band type
/devices/bulk-import	Bulk Import Devices	Drop zone, schema selector, error table
/devices/track	Track Device	Serial search, QR scanner, history timeline
/distributions	Distributions	KPI breakdown cards, status filter, detail modal
/distributions/create	Create Distribution	Device picker, recipient search, notes
/distributions/bulk-upload	Bulk Upload Distribution	File upload, recipient picker, row errors
/defects	Defect Reports	Tabbed by status, filter panel, modal
/defects/create	Create Defect Report	Device search, severity picker, image upload

Route	Page	Key UI Elements
/replacements	Replacements	Active replacement mappings list
/replacements/pending	Pending Replacements	Defects awaiting replacement assignment
/pending-dues	Pending Dues	Financial obligations summary per user
/returns	Returns	Return request list with approval actions
/delivery-confirmations	Delivery Confirmations	Pending distributions for recipient
/replacement-confirmation	Replacement Confirmation	Confirm replacement receipt
/users	Users	Searchable table, create/edit/status modals
/users/hierarchy	User Hierarchy	Visual org chart tree
/approvals	Approvals	Approve/reject queue with routing config
/reports	Reports	Filter form, export to PDF
/backup	Backup	Download backup, scheduler status
/activities	Activity Log	Paginated timeline of all API actions
/notifications	Notifications	Full list with read/unread state
/external-inventory	External Inventory	Items, POs, receipts, movements
/change-requests	Change Requests	Pending requests with review UI
/profile	Profile	View/edit own profile fields
/settings	Settings	Theme, compact mode, notification preferences
/unauthorized	Unauthorized	Role access denied page
*	Not Found	404 page

9.3 Navigation Guard Logic

```
// ProtectedRoute checks in order:  
1. isAuthenticated?      → No → redirect /login  
2. loading?              → Yes → show spinner  
3. isForcedCredentialUpdate? → Yes → redirect /force-update-credentials  
4. allowedRoles check?   → Fail → redirect /unauthorized  
5. → render children
```

9.4 Key UI Interactions

Interaction	Behavior
-------------	----------

Interaction	Behavior
Status badges	Color-coded chips per status value
Detail modals	Open inline — device, distribution, defect info without navigation
Confirmation dialogs	Before destructive actions (cancel distribution, delete device)
Toast notifications	Success/error feedback on all API responses
Debounced search	Input pause → API call with search query param
Compact mode	Tighter padding and reduced font sizes, toggled in settings
QR Scanner	Opens camera in device track page to scan serial number labels
Notification bell	Red badge with unread count; dropdown shows latest 5

10. Error Handling & Edge Cases

10.1 System-Level Errors

Scenario	Behavior
MySQL pool exhaustion	<code>aiomysql</code> raises <code>PoolClosedError</code> ; route returns <code>500</code> with generic message
DB connection failure on startup	<code>init_db()</code> raises; Uvicorn fails to start
File disk write failure	Try/except in upload handler; <code>500</code> with generic message
Backup scheduler crash	Background task cancelled gracefully via <code>asyncio.CancelledError</code> on shutdown
Migration already applied	<code>ALTER TABLE</code> errors silently caught per statement
Token signature invalid	<code>401 Not authenticated</code>

10.2 User-Level Errors

Scenario	Status	Message
Wrong credentials	<code>401</code>	"Invalid email or password"
Inactive/suspended account	<code>403</code>	"Account is not active"
Duplicate MAC or serial	<code>400</code>	Field-specific message
Distributing unowned device	<code>400</code>	"Device X is not in your possession"
Distributing locked device	<code>400</code>	"Device X is in an unconfirmed distribution"
Re-distributing before receipt	<code>400</code>	"Confirm receipt of incoming transfer first"
Distributing defective device	<code>400</code>	"Device X is defective and cannot be transferred"
Wrong hierarchy level	<code>400</code>	Role-specific hierarchy message
Duplicate active defect	<code>400</code>	"Device already has an active defect report"
Invalid file type	<code>400</code>	Extension allowlist message
File magic bytes mismatch	<code>400</code>	"Invalid XLSX/XLS file content"
Confirming payment early	<code>400</code>	"Cannot confirm payment before device is received"
Bulk upload with errors	<code>200</code>	Per-row error list returned; distribution not created

10.3 Fail-Safe Mechanisms

Mechanism	Detail
-----------	--------

Mechanism	Detail
Device holder deferred transfer	Devices move to recipient only after receipt is confirmed, never on distribution creation
Transaction rollback	All DB writes use <code>await db.commit()</code> — uncommitted work is rolled back on exception
Manifest generation failure	If Excel manifest fails, distribution still succeeds; manifest is optional
Auto-return creation failure	If auto-return creation fails after defect approval, status update still succeeds
Idempotent migrations	All <code>ALTER TABLE</code> migration statements are individually wrapped in <code>try/except</code>
Token refresh on 401	Frontend authContext intercepts <code>401</code> , silently refreshes, retries original request
Disputed device lock	Sender cannot redistribute devices while a distribution is in <code>disputed</code> state

11. Setup & Deployment

11.1 Prerequisites

Requirement	Local Dev	Production
Python	3.11+	— (in Docker)
Node.js	18+	— (in Docker)
Docker Desktop	For MySQL	Required
Docker Compose plugin	Optional	Required
Git	Required	Required
Tailscale	Optional	Required

11.2 Local Development Setup (Windows)

Step 1 — Clone the repository:

```
git clone <repo-url> distribution-management-system
cd distribution-management-system
```

Step 2 — Start MySQL via Docker:

```
docker compose up mysql -d
```

Wait ~15 seconds for MySQL to complete its healthcheck.

Step 3 — Set up Python backend:

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Step 4 — Create backend environment file:

Create the file `backend/.env` with the following contents:

```
# Application
APP_NAME=Distribution Management System
APP_VERSION=1.0.0
DEBUG=true
ENVIRONMENT=development
```

```
# Server
HOST=127.0.0.1
PORT=8080

# Database (matches docker-compose.yml defaults)
DB_HOST=localhost
DB_PORT=3306
DB_USER=dms_user
DB_PASSWORD=dms_password
DB_NAME=distribution_management_system

# Security – generate a fresh key before starting
SECRET_KEY=<run: python -c "import secrets; print(secrets.token_urlsafe(64))">
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
CSRF_COOKIE_SECURE=false
ENFORCE_HTTPS=false

# CORS
CORS_ORIGINS=http://localhost:5173
```

Step 5 — Start the backend:

```
python -m uvicorn app.main:app --reload --port 8080
```

On first start, the backend automatically:

- Creates all 19 database tables
- Applies column migrations (idempotent)
- Seeds the super admin account (admin@dms.com)
- Starts the monthly backup scheduler

Backend available at: <http://localhost:8080>

Swagger UI (dev only): <http://localhost:8080/docs>

Step 6 — Set up the frontend:

```
cd ../frontend
npm install
```

Create the file [frontend/.env](#):

```
VITE_API_URL=http://localhost:8080/api
```

Step 7 — Start the frontend:

```
npm run dev
```

Frontend available at: <http://localhost:5173>

Step 8 — Optional: Use PowerShell convenience scripts:

```
.\start.ps1    # starts backend + frontend
.\stop.ps1     # stops both
```

11.3 Default Login Credentials

These credentials are seeded on first startup in development mode.

Role	Email	Password
Super Admin	admin@dms.com	Admin@123
Manager	manager1@dms.com	Manager@123
PDIC Staff	staff1@dms.com	Staff@123
Sub Distributor	subdist1@dms.com	SubDist@123
Operator	operator1@dms.com	Oper@123

⚠ All first-time logins trigger the **Forced Credential Update** screen. Users must set a new email and password before proceeding.

Seed full test data (development only):

```
curl -X POST http://localhost:8080/reset-and-seed
# Creates: 39 users, 200 devices, 50 distributions, 30 defects
```

11.4 Docker Compose — Full Stack

Run the entire stack (MySQL + Backend + Frontend) together:

```
# Build and start all containers
docker compose up -d --build

# View real-time logs
docker compose logs -f backend
```

```
# Restart a service
docker compose restart backend

# Stop all services
docker compose down
```

Container summary:

Container	Port	Purpose
dms-mysql	3306	MySQL 8.4 database
dms-backend	8080	FastAPI API server
dms-frontend	5173	Vite / React SPA

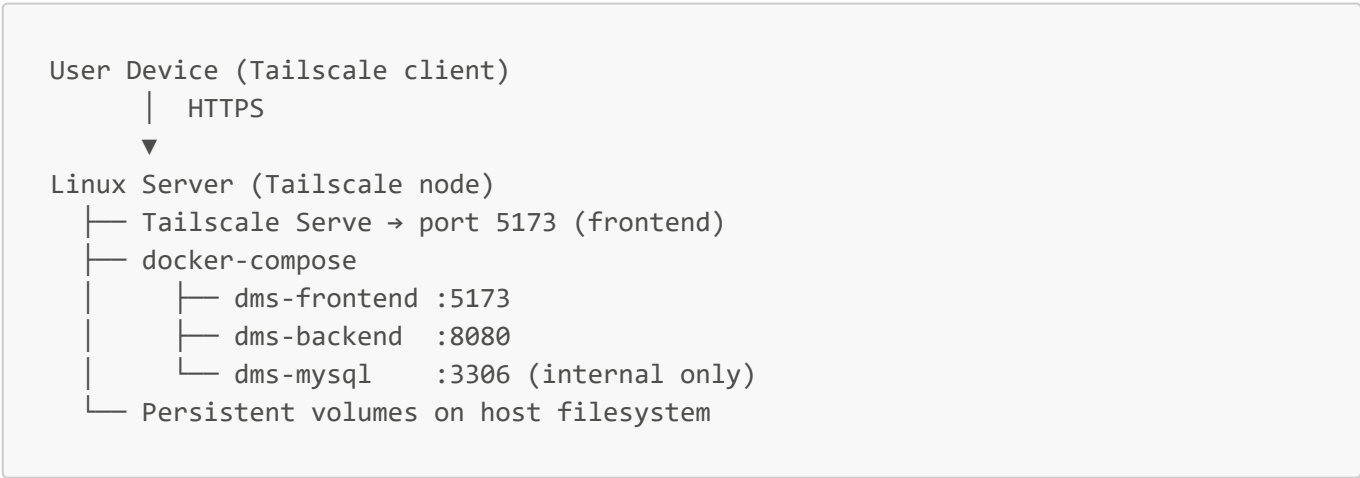
Persistent volumes:

Volume	Contents
mysql_data	All database files
./backend/distribution_manifests	Generated Excel distribution manifests
./backend/monthly_backups	Scheduled backup files
./backend/uploads	Uploaded images, payment bills

11.5 Production Deployment — Headless Server + Tailscale

This section covers deploying the DMS on a Linux server with no GUI, accessible privately via Tailscale VPN.

Architecture:



Step 1 — Prepare the server

```
sudo apt update && sudo apt -y upgrade
sudo apt -y install ca-certificates curl gnupg git ufw jq
```



```
# Firewall baseline
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp      # SSH only
sudo ufw enable
```

Do not expose ports 8080, 5173, or 3306 publicly. All access goes through Tailscale.

Step 2 — Install Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt -y install docker-ce docker-ce-cli containerd.io \
                        docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Log out and back in for the group membership to take effect.

Step 3 — Install and join Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
# Follow the printed URL to authorize the server in your Tailscale admin console

tailscale status  # confirm connected
tailscale ip -4   # note the tailnet IP
```

Step 4 — Deploy application code

```
sudo mkdir -p /opt/dms
sudo chown $USER:$USER /opt/dms
cd /opt/dms
git clone <repo-url> distribution-management-system
cd distribution-management-system
git checkout <release-tag>
```

Step 5 — Configure production environment

Create **backend/.env**:

```
APP_NAME=Distribution Management System
APP_VERSION=1.0.0
DEBUG=False
ENVIRONMENT=production

HOST=0.0.0.0
PORT=8080

DB_HOST=mysql
DB_PORT=3306
DB_USER=dms_user
DB_PASSWORD=<strong-random-db-password>
DB_NAME=distribution_management_system

SECRET_KEY=<64-byte-ur-safe-secret>
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
CSRF_COOKIE_SECURE=true
ENFORCE_HTTPS=false

CORS_ORIGINS=https://<server-hostname>.<tailnet>.ts.net
ADMIN_INITIAL_PASSWORD=<strong-temporary-admin-password>
```

Generate a secure secret key:

```
python3 -c "import secrets; print(secrets.token_urlsafe(64))"
```

Create **frontend/.env.production**:

```
VITE_API_URL=https://<server-hostname>.<tailnet>.ts.net/api
```

This file is **critical**. Without it, the frontend is built with <http://localhost:8080/api> which will not work for remote users.

Step 6 — Optional: Remove MySQL host port exposure

Create **docker-compose.prod.yml**:

```
services:
  mysql:
    ports: []
```

Step 7 — Start services

```
# Standard
docker compose up -d --build

# With production override (recommended)
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Step 8 — Enable HTTPS via Tailscale Serve

```
sudo tailscale serve --https=443 / http://127.0.0.1:5173
sudo tailscale serve status      # verify configuration
sudo systemctl enable --now tailscaled # persist across reboots
```

Users access the application at:

```
https://<server-hostname>.<tailnet>.ts.net
```

Step 9 — Validate go-live

```
# Backend health
curl -sS http://127.0.0.1:8080/health
# Expected: {"status":"healthy"}

# Frontend reachable
curl -I http://127.0.0.1:5173

# Container status
docker compose ps

# Tailscale
tailscale serve status
```

11.6 Adding Users to the System

Technical Access (Tailnet)

1. In Tailscale admin console → **Invite user** by email
2. Assign ACL group (e.g., `group:dms-users`)
3. Ensure ACL allows destination port `443`
4. User accepts invite, installs Tailscale, connects

Application Access (DMS Login)

1. Log in as super admin (`admin@dms.com`)
 2. Navigate to **Users** → **Add User**
 3. Fill in name, email, role, and parent user (where required)
 4. Share credentials securely
 5. User logs in → forced to update email and password on first login
-

11.7 Day-2 Operations

```
# Check status
docker compose ps

# View logs
docker compose logs -f backend
docker compose logs -f mysql

# Restart service
docker compose restart backend

# Update deployment
git fetch --all
git checkout <new-release-tag>
git pull --ff-only
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Backup policy recommendations:

- Daily MySQL dump using `mysqldump` from within the container
 - Daily filesystem sync of `backend/uploads` and `backend/monthly_backups`
 - Minimum 30-day retention
 - Monthly restore test
-

12. Logging & Monitoring

12.1 Logging Layers

The system uses three distinct logging mechanisms:

Audit Logger (app.core.audit)

A dedicated security event logger for sensitive actions. Writes to server stdout and a persistent log file.

Event Key	Trigger
LOGIN_SUCCESS	Successful authentication
LOGIN_FAILED	Wrong credentials
LOGIN_BLOCKED_INACTIVE	Login attempt on inactive account
LOGOUT_SUCCESS	User logout
TOKEN_REFRESH_SUCCESS/FAILED	Token refresh attempt
PASSWORD_CHANGE_SUCCESS/FAILED	Password change
FORCED_CREDENTIAL_ROTATION_COMPLETE	First-login credential update done
USER_DELETE	User account deleted
USER_STATUS_UPDATE	Account status changed
USER_CREDENTIALS_UPDATE	Admin credential reset
DB_RESET	Development database reset (with IP logged)

API Activity Logger (app.core.activity_logger)

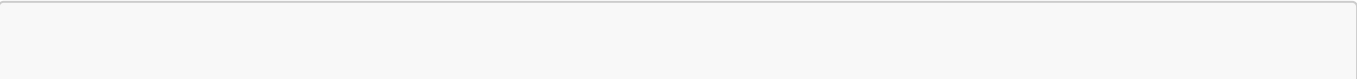
Captures every significant API request via `ApiActivityLoggingMiddleware`. Stored in `api_activity_logs` table.

```
Every API request → middleware runs post-response
→ build_meaningful_activity_description(method, path, status_code)
→ if description exists → write to api_activity_logs
→ if description is None → silently skip (e.g., GET /health)
```

Application Logger (Python standard logging)

Each route module has `logger = logging.getLogger(__name__)`. Unhandled exceptions are captured with `logger.exception()` — full stack traces go to server logs, never to the client.

12.2 Health Check



```
GET /health
→ 200 OK {"status": "healthy"}
```

Used by Docker healthchecks and external uptime monitors.

12.3 Viewing Logs

```
# Container backend logs (follow)
docker compose logs -f backend

# Last 100 lines
docker compose logs backend --tail=100

# MySQL logs
docker compose logs mysql --tail=50

# Activity log in-app
# Navigate to /activities (super_admin or md_director only)
```

12.4 Monitoring Recommendations

Area	Recommendation
Uptime	Poll GET /health every 60s via Uptime Kuma or similar
Log aggregation	Ship Docker logs to Grafana Loki, ELK, or cloud logging
Disk space	Monitor backend/uploads and backend/monthly_backups growth
DB backup validation	Monthly restore test from monthly_backups/
Failed login alerts	Monitor audit log for repeated LOGIN_FAILED events per IP

13. Future Improvements

13.1 Scalability

Area	Recommendation
Database	Migrate from single MySQL container to managed RDS with read replicas
Async jobs	Add Celery + Redis for notification delivery and report generation
File storage	Move uploads to S3-compatible object storage (AWS S3, MinIO)
WebSockets	Replace polling notification pattern with real-time WebSocket push
Caching	Add Redis caching for frequently read, rarely changing data (user hierarchy, routing config)
Load balancing	Add Nginx reverse proxy layer with multiple Uvicorn workers

13.2 Feature Enhancements

Feature	Description
Email notifications	SMTP integration for approval alerts and defect updates
Mobile app	React Native / Flutter app for field operators
QR code generation	Generate printable QR labels at device registration
Geo-tracking	GPS coordinates at time of defect report
SLA management	Automatic escalation when defect statuses exceed configured time thresholds
Advanced analytics	MTTR tracking, defect root cause trends, distribution velocity heatmaps
Inventory forecasting	Low-stock alerts when <code>quantity_on_hand</code> drops below <code>reorder_level</code>
Multi-tenancy	Support multiple independent organizations on one deployment
SSO / OAuth2	SAML/OAuth2 integration for enterprise identity providers
2FA	TOTP-based two-factor authentication for admin and manager accounts

13.3 Operational Improvements

Area	Recommendation
CI/CD	GitHub Actions → build Docker images → push to registry → deploy via SSH
Database migrations	Migrate from inline <code>ALTER TABLE</code> to Alembic for version-controlled schema changes
Secrets management	Replace <code>.env</code> file secrets with HashiCorp Vault or AWS Secrets Manager

Area	Recommendation
Automated cloud backups	Schedule upload of <code>monthly_backups/</code> to S3 or similar
Key rotation	Automate <code>SECRET_KEY</code> rotation with a grace period for existing tokens
Dependency scanning	Add <code>pip-audit</code> and <code>npm audit</code> to CI pipeline

Document Information

Field	Value
System Name	Distribution Management System
Version	1.0.0
Backend	FastAPI · Python 3.11+ · MySQL 8.4
Frontend	React 18 · Vite 6 · Tailwind CSS 3
Deployment	Docker Compose · Tailscale
Document Date	April 2026
Classification	Confidential

*This document was generated from direct source code analysis of the production codebase.
All workflows, API endpoints, database schemas, and security controls reflect the actual implementation.*